

Minibot: guía de referencia rápida

Mauricio Matamoros

22 de mayo de 2026

1. Guía rápida:

El firmware está escrito en C/C++ y requiere del IDF framework de Espressif que está basado en FreeRTOS. La estructura de directorios se basa en la plantilla de aplicación de Espressif en [Se](#) aconseja revisar la documentación del [Espressif IDF](#) para saber más sobre cómo instalar la plataforma.

La guía rápida asume que se compila desde una distribución linux basada en debian. Aunque es posible desarrollar en sistemas windows, tendrá que revisar la documentación del IDF para saber cómo llevar a cabo una compilación cruzada.

1.1. Prerrequisitos

1. Instale los siguientes paquetes

```
sudo apt install git wget flex bison gperf python3 python3-pip python3-setuptools cmake  
sudo apt install ninja-build ccache libffi-dev libssl-dev dfu-util libusb-1.0-0
```

2. Descargue e instale el Espressif esp-idf versión 5.5 en ~/esp32

```
mkdir ~/esp32  
cd ~/esp32  
git clone -b v5.5 --depth 1 https://github.com/espressif/esp-idf.git  
cd esp-idf  
./install.sh
```

3. Importe el idf framework en la terminal. Esto debe hacerse *SIEMPRE* antes de intentar compilar un proyecto para el ESP32.

```
source export.sh
```

4. Si no lo ha hecho, clone el repositorio con el framework del minibot

```
cd ~/esp32  
git clone --depth 1 https://github.com/kyordhel/minibot-firmware-esp32.git minibot  
cd minibot
```

1.2. Configuración y compilación

Después de importar el ESP IDF ejecute los siguientes comandos.

```
idf.py set-target esp32s3  
idf.py menuconfig
```

ERROR: NO SE ENCONTRÓ EL ARCHIVO idf.py

Ya sabía que esto iba a pasar. Usted olvidó importar el IDF de nuevo.

```
source /ruta/al/esp-idf/export.sh
```

```

[Top]
Espressif IoT Development Framework Configuration
Build type --->
Bootloader config --->
Security features --->
Application manager --->
Boot ROM Behavior --->
Serial flasher config --->
Partition Table --->
Minibot Configuration --->
Compiler options --->
Component config --->
[ ] Make experimental features visible

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [I] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(a) Ventana principal

```

[Top] -> Minibot Configuration
Espressif IoT Development Framework Configuration
I2C Bus --->
Wireless configuration --->
Base configuration --->
Sensors configuration --->

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [I] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(b) Configuraciones del Minibot

```

[Top] -> Minibot Configuration -> Wireless configuration
Espressif IoT Development Framework Configuration
WiFi operation mode (Wireless Access Point) --->
(Minibot) SSID of the wireless network
(12345678) Password for the wireless network
Security (encryption) of the network (WPA/WPA2) --->
(192.168.1.1) IP address
(255.255.255.0) Network mask
(192.168.1.1) Gateway IP address
(192.168.1.1) DNS server IP address
(minibot.local) Hostname

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [I] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(c) Configuración de la red inalámbrica

```

[Top] -> Minibot Configuration -> Wireless configuration -> WiFi operation mode
Espressif IoT Development Framework Configuration
( ) Wireless Access Point
(X) Client device

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [I] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(d) Cambio a modo cliente

```

[Top] -> Minibot Configuration -> Wireless configuration -> WiFi operation mode (Client device)
Espressif IoT Development Framework Configuration
WiFi operation mode (Client device) --->
(Minibot) SSID of the wireless network
(12345678) Password for the wireless network
(192.168.1.1) IP address
(255.255.255.0) Network mask
(192.168.1.1) Gateway IP address
(192.168.1.1) DNS server IP address
(minibot.local) Hostname
SSID of the wireless network (string)
Minibot

```

(e) Conectar a la red Minibot

```

[Top] -> Minibot Configuration -> Wireless configuration -> WiFi operation mode (Client device)
Espressif IoT Development Framework Configuration
WiFi operation mode (Client device) --->
(Minibot) SSID of the wireless network
(12345678) Password for the wireless network
(I) IP address
( ) Network mask
( ) Gateway IP address
( ) DNS server IP address
(minibot.local) Hostname

[Space/Enter] Toggle/enter [ESC] Leave menu [S] Save
[O] Load [?] Symbol info [I] Jump to symbol
[F] Toggle show-help mode [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

(f) Configuración para obtención de IP automática via DHCP

Figura 1: configuración del ESP IDF

El primer comando instruye al IDF indicando que el integrado objetivo es un ESP32-S3. Si usted cuenta con otro integrado, especifique el tipo de integrado a usar. El segundo comando abre la herramienta de configuración del ESP IDF. Verá una ventana como la de la [Figura 1a](#).

A continuación, navegue dentro de la herramienta de configuración a *Minibot configuration* $\Rightarrow$ *Wireless configuration* $\Rightarrow$ *WiFi operation mode* y establezca el modo de conexión como cliente para que el Minibot se conecte a una red inalámbrica existente; tal como se muestra en las [Figuras 1a](#) a [1d](#).

El siguiente paso consiste en establecer el nombre de la red y la contraseña de la misma (véase [Figura 1e](#)). Si conoce la configuración de su red, se puede establecer una IP estática para facilitar el acceso al Minibot. Sin embargo, si prefiere usar la asignación dinámica de direcciones IP vía DHCP de su red, es necesario borrar las configuraciones de la misma como se ilustra en la [Figura 1f](#).

Finalmente, compile la solución con el siguiente comando

```
idf.py build
```

1.3. Carga en el integrado (flasheo)

Después de importar el ESP IDF conecte el ESP32 del minibot y ejecute el siguiente comando.

```
df.py flash
```

Si lo desea, puede habilitar el monitor para observar el proceso de arranque del Minibot con el comando tras el flasheo

```
df.py flash monitor
```

El minibot siempre imprime la dirección IP utilizada tras una conexión exitosa.

2. Prueba de comportamientos con PumaSimBot

2.1. Prerrequisitos

1. Instale los siguientes paquetes

```
sudo apt install build-essential cmake libboost-all-dev libncurses-dev
sudo apt install python3 python3-tk python3-numpy
```

2. Si no lo ha hecho, clone el repositorio del Pumasimbot

```
cd ~/
git clone --depth 1 --branch ft/minibot https://github.com/kyordhel/pumasimbot.git
cd pumasimbot
```

2.2. Compilación

Edite el archivo `src/motion_planner/GoTo_State_Machine.cpp`; particularmente la línea 605 donde deberá incluir la dirección IP del minibot.

```
if( virbot_using_real_robot() ){
    // Write down IP address of the robot here.
    minibot_connect("192.168.1.102", 9000, 10);
    if(!minibot_connected()) return -1;
    float v; int p;
    get_battery_charge(&v, &p);
    fprintf(stderr, "Battery charge: %0.1fV (%d%%)", v, p);
}
```

Por último, ejecute los siguientes comandos para compilar el simulador

```
mkdir -p build
cd build
cmake .. -DTCP_CLIPS60_FETCH_FROM_GIT=1 -DMINIBOT_FETCH_FROM_GIT=1 \
        -DMINIBOT_TCP_API_FETCH_FROM_GIT=1
make
cd ..
```

2.3. Ejecución y prueba

La ejecución del simulador es relativamente sencilla. Basta con usar el siguiente comando

```
xterm -hold -e "cd bin/gui && python3 pumasimbot.py 1" &
```

3. Interfaz web

La interfaz web permite monitorear el estado del minibot y enviar comandos rápidos. Se accede directamente en la dirección IP asignada al minibot; por ejemplo <http://192.168.1.102>; o bien 192.168.1.102:80.

La interfaz web es minimalista y autoexplicativa. En ella podrá observar el estado de los sensores de proximidad (distancia/obstáculos), enviar comandos mv, establecer el PWM de los motores, y ejecutar comandos rápidos de movimiento como giros a la izquierda, derecha, avanzar y retroceder. Además se indica la posición estimada de la fuente de luz más intensa y el nivel de carga de la batería.

Todas las peticiones viajan por GET, por lo que es posible obtener el estado del robot en formato JSON con una llamada a `minibot?cmd=<cmd>&args=<args>`. Por ejemplo <http://192.168.1.102/minibot?cmd=sensors> devolverá un JSON con la información de todos los sensores del robot. De forma similar <http://192.168.1.102/minibot?cmd=mv&args=1.5709%200.1> hará que el robot gire 90° a la izquierda y avance diez centímetros.



Figura 2: Interfaz web de control

Sin soporte para SSH

Muchos navegadores reemplazan automáticamente las direcciones de web standard `http` por direcciones encriptadas en web seguro `https` que opera en el puerto 433.

La implementación actual sólo responde a peticiones realizadas en el puerto 80 para `http` y no incluye certificados de seguridad. Si obtiene un error 404, verifique que la dirección sea la correcta.

Tabla 3: Comandos soportados

Código	Comando	Argumentos	Retorno	Descripción
0×0100	mv	ff	fff	ViRbot mv: distancia (metros) y angulo (radianes)
0×1000	Get PWM	—	ffff	Devuelve los valores de PWM establecidos en cada rueda
0×1200	Set PWM	ff	—	PWM para las ruedas izquierda y derecha
0×1400	Set PWM	ffff	—	Establece los valores de PWM para las cuatro ruedas
0×2000	Get speed	—	ffff	No implementado
0×2200	Set speed	ff	—	No implementado
0×2400	Set speed	ffff	—	No implementado
0×4000	Read all	—	Varía	Lee todos los sensores
0×4100	Read batt	—	ff	Lee el estado de la batería
0×4200	Read dist	—	Varía	Lee el anillo de sensores de distancia como LIDAR
0×4300	Read floor	—	ffff	Lee los sensores de piso
0×4400	Read light	—	$f \times 16$	Lee el anillo de sensores de luz
0×0000	STOP	—	—	Cancela cualquier operación de movimiento

Nota: f implica un valor flotante de 32 bits.

4. Servidor tcp

El servidor TCP es el canal predeterminado para operar el minibot de forma rápida y eficiente. La conexión se realiza vía el puerto 9000 en la dirección IP asignada al minibot; por ejemplo [192.168.1.102:9000](#). El protocolo es de dos vías: solicitud-respuesta. Los datagramas respectivos se muestran en las tablas [Tablas 1 y 2](#).

Tabla 1: Datagrama base de una solicitud

Offset	Contenido	Tipo	Tamaño	Descripción
0	Id	uint32	4 bytes	Identificador único del mensaje.
4	Tamaño	uint32	4 bytes	Tamaño del mensaje en bytes.
8	Comando	uint32	4 bytes	Comando a ejecutar.
12	Argumentos	—	Varía	Opcional. Argumentos del comando.

Tabla 2: Datagrama base de una respuesta

Offset	Contenido	Tipo	Tamaño	Descripción
0	Id	uint32	4 bytes	Identificador único del mensaje.
4	Tamaño	uint32	4 bytes	Tamaño del mensaje en bytes.
8	Comando	uint32	4 bytes	Comando a ejecutar.
12	Resultado	bool	1 byte	¿Ejecución exitosa?
13	Argumentos	—	Varía	Opcional. Valores de retorno.

Cualquier paquete mal formado, incluyendo paquetes de menos de 12 bytes, o aquellos donde el tamaño no corresponda a la carga son descartados por el sistema y no generan respuesta. Un breviarío de los comandos aceptados se muestran en la [Tabla 3](#):

4.1. Comando mv

El robot gira y avanza.

Código 0×0100

Argumentos

- | | | | |
|---|---------|----------------|----------------------------------|
| 1 | 4 bytes | Punto flotante | La distancia a avanzar en metros |
| 2 | 4 bytes | Punto flotante | El ángulo a girar en radianes |

Valores devueltos

- | | | | |
|---|---------|----------------|--|
| 1 | 4 bytes | Punto flotante | Desplazamiento Δx en metros |
| 2 | 4 bytes | Punto flotante | Desplazamiento Δy en metros |
| 3 | 4 bytes | Punto flotante | Ángulo rotado $\Delta\theta$ en radianes |

4.2. Comando GetPWM

Obtiene los ciclos de trabajo para el PWM de los motores del robot.

Código 0×1000

Argumentos

Ninguno

Valores devueltos

- | | | | |
|---|---------|----------------|--|
| 1 | 4 bytes | Punto flotante | Ciclo de trabajo del motor izquierdo en el intervalo $[-1, 1]$ |
| 2 | 4 bytes | Punto flotante | Ciclo de trabajo del motor derecho en el intervalo $[-1, 1]$ |
| 3 | 4 bytes | Punto flotante | Ciclo de trabajo del motor frontal en el intervalo $[-1, 1]$ |
| 4 | 4 bytes | Punto flotante | Ciclo de trabajo del motor trasero en el intervalo $[-1, 1]$ |

4.3. Comando SetPWM2

Establece los ciclos de trabajo para el PWM de los motores laterales del robot.

Código 0×1200

Argumentos

- | | | | |
|---|---------|----------------|--|
| 1 | 4 bytes | Punto flotante | Ciclo de trabajo del motor izquierdo en el intervalo $[-1, 1]$ |
| 2 | 4 bytes | Punto flotante | Ciclo de trabajo del motor derecho en el intervalo $[-1, 1]$ |

Valores devueltos

Ninguno

4.4. Comando SetPWM4

Establece los ciclos de trabajo para el PWM de todos motores del robot.

Código 0×1400

Argumentos

- | | | | |
|---|---------|----------------|--|
| 1 | 4 bytes | Punto flotante | Ciclo de trabajo del motor izquierdo en el intervalo $[-1, 1]$ |
| 2 | 4 bytes | Punto flotante | Ciclo de trabajo del motor derecho en el intervalo $[-1, 1]$ |
| 3 | 4 bytes | Punto flotante | Ciclo de trabajo del motor frontal en el intervalo $[-1, 1]$ |
| 4 | 4 bytes | Punto flotante | Ciclo de trabajo del motor trasero en el intervalo $[-1, 1]$ |

Valores devueltos

Ninguno

4.5. Comando ReadAll

Lee todos los sensores en el siguiente orden: batería, sensores de piso, sensores de luz, sensores de obstáculos.

Código 0×4000

Argumentos

Ninguno

Valores devueltos

1	4 bytes	Punto flotante	Voltaje de la batería
2	1 byte	Entero signado	Porcentaje de carga de la batería
3	4 bytes	Punto flotante	Valor normalizado del sensor de piso 0
4	4 bytes	Punto flotante	Valor normalizado del sensor de piso 1
5	4 bytes	Punto flotante	Valor normalizado del sensor de piso 2
6	4 bytes	Punto flotante	Valor normalizado del sensor de piso 3
7	4 bytes	Punto flotante	Ángulo en radianes del primer sensor sensor de luz
8	4 bytes	Punto flotante	Valor normalizado del primer sensor de luz
...			
21	4 bytes	Punto flotante	Ángulo en radianes del octavo sensor sensor de luz
22	4 bytes	Punto flotante	Valor normalizado del octavo sensor de luz
23	2 bytes	Entero no signado	Número de tuplas reportadas
24	4 bytes	Punto flotante	Ángulo en radianes del primer sensor de obtáculos
...			
$22 + 2n$	4 bytes	Punto flotante	Ángulo en radianes del n-ésimo sensor de obtáculos
$23 + 2n$	4 bytes	Punto flotante	Valor normalizado del n-ésimo sensor de obtáculos

4.6. Comando ReadBatt

Lee la carga de la batería. Cuando el comando falla los valores devueltos son -1.

Código 0×4100

Argumentos

Ninguno

Valores devueltos

1	4 bytes	Punto flotante	Voltaje de la batería en voltios, sensado por la tarjeta controladora de los motores
2	1 byte	Entero signado	Porcentaje de carga de la batería en nivel operativo proporcional al rango [5.5, 7.2]V equivalente a [0, 100] %

4.7. Comando ReadDist

Lee el anillo de sensores de distancia para detectar obstáculos. Cuando el comando falla los valores devueltos son -1.

Código 0×4200

Argumentos

Ninguno

Valores devueltos

1	2 bytes	Entero no signado	Número de tuplas reportadas
2	4 bytes	Punto flotante	Ángulo en radianes del primer sensor de obtáculos.
3	4 bytes	Punto flotante	Valor normalizado del primer sensor de obtáculos en el rango [0, 1]
4	4 bytes	Punto flotante	Ángulo en radianes del segundo sensor sensor de obtáculos
5	4 bytes	Punto flotante	Valor normalizado del segundo sensor de obtáculos en el rango [0, 1]
...			
$2n$	4 bytes	Punto flotante	Ángulo en radianes del n-ésimo sensor de obtáculos.
$2n + 1$	4 bytes	Punto flotante	Valor normalizado del n-ésimo sensor de luz en el rango [0, 1]

4.8. Comando ReadFloor

Lee los sensores de piso en el orden indicado en la configuración del minibot. Cuando el comando falla los valores devueltos son -1.

Código 0×4300

Argumentos

Ninguno

Valores devueltos

1	4 bytes	Punto flotante	Valor normalizado del sensor infrarrojo 0 en el rango $[0, 1]$
2	4 bytes	Punto flotante	Valor normalizado del sensor infrarrojo 1 en el rango $[0, 1]$
3	4 bytes	Punto flotante	Valor normalizado del sensor infrarrojo 2 en el rango $[0, 1]$
4	4 bytes	Punto flotante	Valor normalizado del sensor infrarrojo 3 en el rango $[0, 1]$

4.9. Comando ReadLight

Lee los sensores de luz de la torreta. Los sensores ignorados devuelven valores -1.

Código 0×4400

Argumentos

Ninguno

Valores devueltos

1	4 bytes	Punto flotante	Ángulo en radianes del primer sensor sensor de luz.
2	4 bytes	Punto flotante	Valor normalizado del primer sensor de luz en el rango $[0, 1]$
3	4 bytes	Punto flotante	Ángulo en radianes del segundo sensor sensor de luz
4	4 bytes	Punto flotante	Valor normalizado del segundo sensor de luz en el rango $[0, 1]$
...			
15	4 bytes	Punto flotante	Ángulo en radianes del octavo sensor sensor de luz
16	4 bytes	Punto flotante	Valor normalizado del octavo sensor de luz en el rango $[0, 1]$

4.10. Comando STOP

Cancela la ejecución de un comando mv y para los motores.

Código 0×0000

Argumentos

Ninguno

Valores devueltos

Ninguno